

Transformer (上): Seq2seq 与 Encoder

李宏毅《机器学习》2025 第六节：从序列到序列模型到 Transformer 编码器



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅机器学习深度学习课程 (Bilibili)

时长：约 33 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=15>

目录

1 Transformer 从 Seq2seq 开始	3
1.1 和前面 Self-attention 的关系	4
1.2 本章小结	4
2 为什么许多任务都能被看成 Seq2seq	4
2.1 语音翻译：绕开中间文字的瓶颈	4
2.2 问答统一视角：把 NLP 任务改写成 QA	5
2.3 树结构也能线性化	6
2.4 Seq2seq 不是万能最优解	7
2.5 本章小结	7
3 从传统 Seq2seq 到 Transformer	8
3.1 Encoder 和 BERT 的关系	9
3.2 本章小结	9
4 Transformer Encoder：一排向量到另一排向量	9
4.1 为什么不能只写一个 Self-attention	10
4.2 位置编码不可省	11
4.3 本章小结	11
5 一个 Encoder Block 的内部结构	11
5.1 Layer Normalization	11
5.2 Feed Forward Network 的角色	12
5.3 本章小结	13
6 对照原始 Transformer Encoder 图	13
6.1 Multi-head 的意义	14
6.2 为什么这里讲 LayerNorm 而不是 BatchNorm	14
6.3 本章小结	14
7 Normalization 位置不是唯一答案	14
7.1 Pre-norm 与 Post-norm 的直觉差异	15
7.2 本章小结	16
8 总结与延伸	16
8.1 拓展阅读	17

1 Transformer 从 Seq2seq 开始

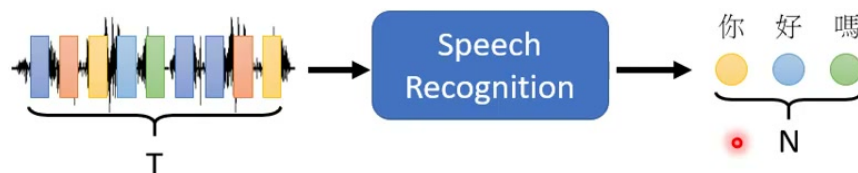
这讲课的入口不是一上来写 Transformer 的公式，而是先问一个更基础的问题：什么样的任务需要模型“读入一个序列，再产生另一个序列”？Transformer 本质上就是一个 sequence-to-sequence model，通常简称为 Seq2seq。这里的 sequence 不一定是文字序列，也可以是声音讯号、影像切出来的 patch、标签序列、树结构展开后的符号序列，甚至是模型自己逐步生成的一串动作。

Seq2seq 的关键不是“输入和输出都是序列”这么表面，而是输出长度通常不能由输入长度直接决定。语音辨识是最典型的例子：输入是一段长度为 T 的声音讯号，输出是一串中文字或 token，长度为 N 。 T 和 N 当然有关，但不能只靠 T 推出 N ；模型必须听懂内容以后，自己决定要输出几个 token。

Sequence-to-sequence (Seq2seq)

Input a sequence, output a sequence

The output length is determined by model.



2

图 1: Seq2seq 的基本形式：输入是一段声音序列，输出是一段文字序列，输出长度由模型根据内容决定。²

如果把输入序列记作 $X = (x_1, \dots, x_T)$ ，输出序列记作 $Y = (y_1, \dots, y_N)$ ，Seq2seq 学到的是条件分布

$$P(Y | X).$$

在自回归生成的 decoder 中，这个分布通常会拆成

$$P(Y | X) = \prod_{t=1}^N P(y_t | y_{<t}, X).$$

其中：

²视频画面时间区间：约 02:10–02:30；字幕对应“输入的声音讯号它的长度是大 T”“并没有办法一知道输出长度 N 一定是多少”“由机器自己决定”。

- X 是输入序列，可以是声音 frame、文字 token、影像 patch 等；
- Y 是输出序列，长度 N 往往由模型生成到结束符号时决定；
- $y_{<t}$ 表示已经产生的前缀，模型生成第 t 个 token 时会参考前面已经生成的内容。

Seq2seq 的核心能力

Seq2seq 不是把每个输入位置逐一对应到一个输出位置，而是把完整输入序列当作条件，再一步一步产生输出序列。它适合处理输入输出长度不固定、对齐关系不明显、输出还需要符合语言或结构约束的任务。

1.1 和前面 Self-attention 的关系

前两讲已经介绍过 self-attention：它擅长把一个向量集合或向量序列转换成另一组向量表示。Transformer 把 self-attention 放进 encoder 和 decoder 里面，形成更大的 Seq2seq 系统。也就是说，self-attention 是 Transformer 的局部计算机制；Seq2seq 则是 Transformer 要解决的整体任务形态。

这个区分很重要。只会写 attention 公式，不等于理解 Transformer；理解 Transformer，也不能只停在“输入 token 之间互相看”。真正的模型结构还要回答：如何编码整段输入？如何逐步产生输出？如何把位置信息、残差连接、归一化和全连接网络组合成可训练的深层架构？

1.2 本章小结

Seq2seq 把机器学习任务从“固定维度输入到固定类别或数值”推广到“序列输入到序列输出”。Transformer 是 Seq2seq 模型的一种强大实现，而它内部的主要构件就是 self-attention、残差连接、LayerNorm、Feed Forward 和位置编码。

2 为什么许多任务都能被看成 Seq2seq

老师用大量例子说明：Seq2seq 像一把瑞士刀，很多表面上不像“翻译”的任务，只要能把输入和输出都写成序列，就可以硬套成 Seq2seq。这个观点的价值不在于鼓励所有任务都用同一个模型暴力解决，而在于帮助我们看到“输出格式”本身也可以由模型学习。

2.1 语音翻译：绕开中间文字的瓶颈

语音辨识通常先把声音转成文字，再让人或另一个模型理解文字。但如果某种语言的文字系统不普及，或者使用者看不懂转写出来的字，单纯 ASR 并不能完成沟通任务。闽南语例子里，机器把台语声音辨识成“母汤”这类文字，听得懂台语的人可能知道意思，但一般使用者未必能理解；如果模型能直接输出普通话“不可”“不行”，任务目标就更接近真实需求。

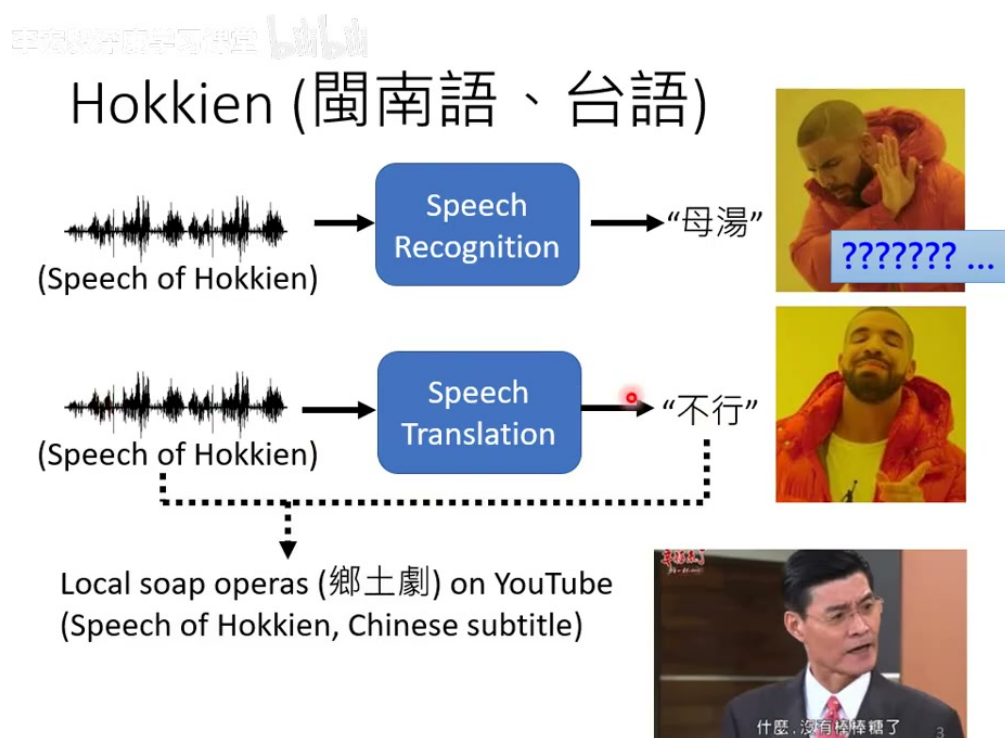


图 2: 同一段闽南语声音可以走“语音辨识”或“语音翻译”两条路径; 后者直接输出目标语言, 避免把不普及的书写形式当作最终结果。⁴

这类任务凸显了 Seq2seq 的一个实用优点: 中间表示不一定要是人类可读的文字。模型可以从声音直接到目标语言文字, 也可以从文字直接到语音, 甚至从一种模态直接到另一种模态。只要训练资料提供输入输出配对, Seq2seq 就有机会学习这个映射。

端到端不是总是更好

直接 speech translation 可以减少错误传播, 但也会吃掉中间监督信号。如果 ASR 资料很多、翻译资料很少, 分阶段系统可能更容易训练; 如果中间文字本身不可靠或不适合用户, 端到端模型反而更自然。模型设计要看资料、任务目标和错误容忍度。

2.2 问答统一视角: 把 NLP 任务改写成 QA

很多自然语言处理任务也可以被包装成问答。给模型一段 context, 再给一个 question, 模型产生 answer。看似不同的任务, 例如翻译、摘要、自然语言推理、情感分析, 都可以改写成类似格式: 问题描述任务, 背景文本提供条件, 答案就是任务输出。

⁴ 视频画面时间区间: 约 05:15–05:55; 字幕对应“台语的文字并不是一般人能够看得懂”“直接输出的是同样意思的中文的句子”。

Most Natural Language Processing applications ...

Question	Context	Answer
What is a major importance of Southern California in relation to California and the US?	...Southern California is a major economic center for the state of California and the US...	major economic center
What is the translation from English to German?	Most of the planet is ocean water.	Der Großteil der Erde ist Meerwasser
What is the summary?	Harry Potter star Daniel Radcliffe gains access to a reported £320 million fortune ...	Harry Potter star Daniel Radcliffe gets £320M fortune...
Hypothesis: Product and geography are what make cream skimming work. Entailment , neutral, or contradiction?	Premise: Conceptually cream skimming has two basic dimensions – product and geography.	Entailment
Is this sentence positive or negative?	A stirring, funny and finally transporting re-imagining of Beauty and the Beast and 1930s horror film.	positive

decaNLP

8

图 3: decaNLP 的例子展示了 QA 统一视角：翻译、摘要、推理、情感分析等任务都能写成 question-context-answer 的序列到序列格式。⁶

这个统一视角对现代大模型尤其重要。今天的 instruction following、prompt-based learning、text-to-text framework，背后都有类似思想：把任务说明和输入资料放进一个序列，再让模型生成目标序列。任务的边界从模型结构转移到输入输出格式上，模型本身可以保持相对统一。

统一格式的好处

当多个任务都被写成“读入一串 token，输出一串 token”，模型可以共享同一套参数和训练流程。不同任务之间的知识可能互相迁移，例如阅读理解训练帮助摘要，翻译训练帮助语义对齐，分类任务则可以被看成只输出很短答案的生成任务。

2.3 树结构也能线性化

句法分析表面上输出的是一棵树，不是序列。老师的例子是句子“deep learning is very powerful”：模型需要判断“deep learning”是名词片语，“very powerful”是形容词片语，再和“is”组成动词片语，最后得到完整句子结构。

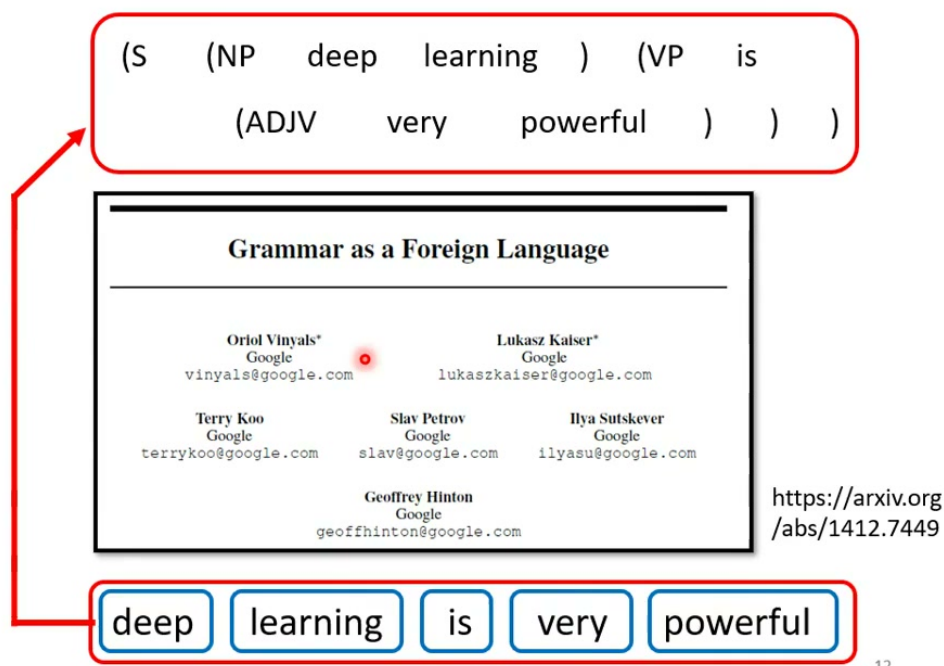
树不是序列，但可以写成括号表达式，例如

$(S (NP \text{ deep learning}) (VP \text{ is } (ADJV \text{ very powerful})))$.

一旦树被线性化，它就变成模型可以逐 token 产生的输出序列。模型不必直接输出一个复杂数据结构，而是输出一种能被解析回树的符号串。

⁶视频画面时间区间：约 12:05–12:35；字幕对应“很多 natural language processing 的任务都可以想成是 question answering 的任务”。

Seq2seq for Syntactic Parsing



12

图 4: 句法分析可以把树状输出线性化成括号序列; Seq2seq 因而能处理看起来不是序列输出的结构预测任务。⁸

这种技巧的意义不止于 parsing。很多结构化预测都可以被“序列化”: 多标签分类可以输出标签序列, 目标检测可以输出类别与 bounding box 序列, 程序生成可以输出代码 token, 数学推理可以输出证明步骤。Seq2seq 的强大之处正在于它把“输出空间的形状”交给 token 序列承载。

2.4 Seq2seq 不是万能最优解

老师反复强调, Seq2seq 可以硬做很多事情, 但不代表每件事情都应该这样做。任务专用模型常常更高效、更可解释、更容易满足约束。例如目标检测若直接生成 box 序列, 需要额外处理重复框、顺序、坐标合法性等问题; 专门的 detection architecture 会把这些先验写进模型结构中。

瑞士刀比喻的正确读法

Seq2seq 是一个通用建模语言: 它让我们知道“很多任务原则上可以统一”。但工程上仍要比较资料量、速度、准确率、约束满足、错误分析和部署成本。统一模型提供可能性, 专门模型提供归纳偏置。

2.5 本章小结

语音翻译、问答、摘要、情感分析、句法分析、多标签分类和目标检测都可以用 Seq2seq 描述。这个统一视角让我们从“为每个任务设计一个输出头”转向“把任务输出写成可生成的序列”。但通

⁸视频画面时间区间: 约 19:35–19:55; 画面展示括号化的 parsing sequence 与相关论文, 字幕讨论“sequence to sequence model 居然可以硬做到”。

用性不等于最优性，实际建模仍需要考虑任务结构和资料条件。

3 从传统 Seq2seq 到 Transformer

Seq2seq 模型的历史早于 Transformer。2014 年左右，基于 RNN/LSTM 的 encoder-decoder 已经被用于机器翻译：encoder 读完整个输入句子，decoder 再逐步产生输出句子。那时的模型相对朴素，长距离依赖和并行训练都比较困难。

Transformer 的突破在于，它用 attention 取代 RNN 的主要序列建模工作。输入不再必须按时间步一个接一个递推；每个位置可以通过 self-attention 直接参考其他位置。对长序列、并行训练和大规模资料而言，这个结构更适合现代硬件，也更容易堆深。

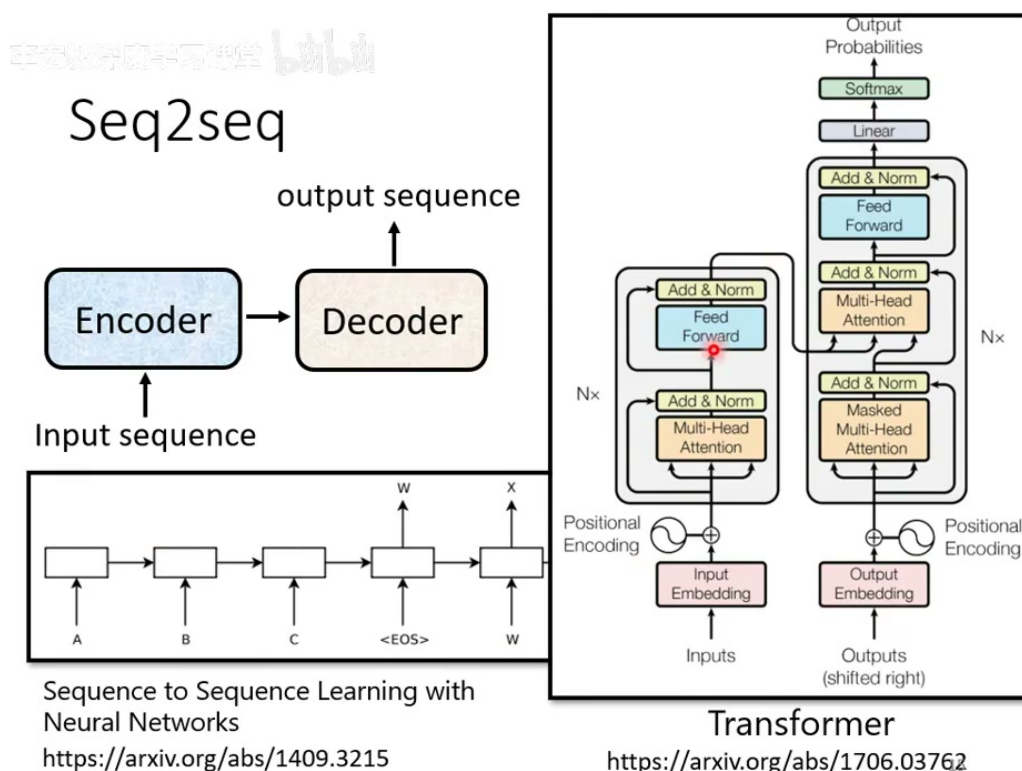


图 5: 传统 Seq2seq 和 Transformer 都是 encoder-decoder 架构；Transformer 论文中的彩色模块把 attention、Add & Norm 和 Feed Forward 组织成可堆叠结构。¹⁰

从系统角度看，Seq2seq 可以抽象为

$$\text{Encoder} : X \mapsto H, \quad \text{Decoder} : (H, y_{<t}) \mapsto y_t.$$

其中：

- $X = (x_1, \dots, x_T)$ 是输入序列；
- $H = (h_1, \dots, h_T)$ 是 encoder 输出的上下文文化表示；

¹⁰视频画面时间区间：约 23:35–24:00；字幕对应“14 年就有 sequence to sequence model 用在翻译”“今天主角也就是 Transformer”“有 encoder 架构，有 decoder 架构”。

- decoder 在生成每个输出 token 时，会参考 H 和已经生成的输出前缀。

本讲后半主要展开 encoder。decoder 的细节会留到下一讲，所以这里先把 encoder 看清楚：它接收一排向量，输出另一排同长度向量；每个输出向量都应该包含对整个输入序列的理解。

3.1 Encoder 和 BERT 的关系

老师在开头用 BERT 暗示 Transformer 与后续模型的关系。严格说，BERT 可以理解为主要使用 Transformer encoder 的模型：它不做自回归输出序列生成，而是把输入序列编码成上下文文化表示，再用这些表示完成遮蔽词预测、句子关系判断或下游任务微调。

因此，学 Transformer encoder 不只是为了完整 Seq2seq，也是在为理解 BERT、现代 embedding model、文本分类模型和许多视觉 Transformer 打基础。encoder 的问题是“如何得到好的表示”；decoder 的问题是“如何根据表示生成东西”。

3.2 本章小结

Transformer 继承了 Seq2seq 的 encoder-decoder 框架，但内部计算从 RNN 递推转向 self-attention 和可并行堆叠的 block。本讲先聚焦 encoder：输入一排向量，输出一排带上下文信息的向量。

4 Transformer Encoder：一排向量到另一排向量

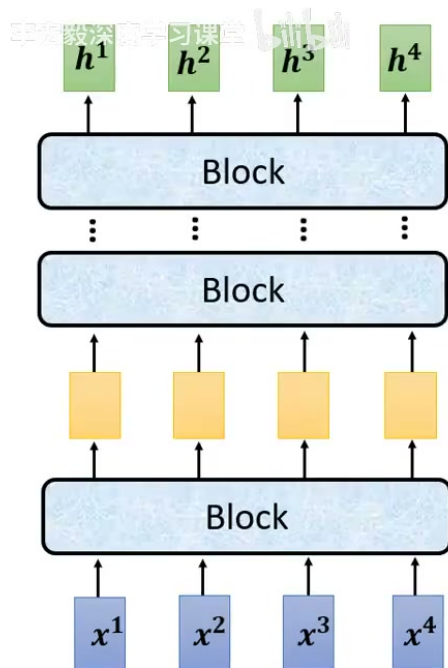
在前面 self-attention 课程里，已经讨论过“输入是一组向量，输出也是一组向量”的问题。Transformer encoder 正是这种问题的标准答案之一。给定输入 token 的 embedding

$$x^1, x^2, \dots, x^T,$$

encoder 输出

$$h^1, h^2, \dots, h^T.$$

输出长度和输入长度相同，但每个 h^i 已经不只是 x^i 的局部变换，而是通过 self-attention 吸收了其他位置的信息。



18

图 6: Transformer encoder 由多个 block 堆叠而成; 每个 block 都把一排向量变成另一排向量, 最后得到上下文表示 $h^1, \dots, h^T$ 。¹²

4.1 为什么不能只写一个 Self-attention

一个 self-attention 层能让不同位置交换信息, 但 Transformer encoder 不是只把 self-attention 单独摆在那里。原因有三点:

- self-attention 主要做跨位置的信息混合, 但每个位置内部还需要非线性变换;
- 深层网络训练需要残差连接来保留梯度通路;
- 中间表示的尺度需要归一化, 否则堆叠很多层后训练容易不稳定。

因此, 一个 encoder block 通常由两类子层组成: multi-head self-attention 和 position-wise feed-forward network。每个子层外面再接 residual connection 和 layer normalization。原始 Transformer 论文图里把这些合称为 Add & Norm。

Position-wise Feed Forward 是什么

Feed Forward Network 不是在不同 token 之间交换信息, 而是对每个位置的向量各自做同一组全连接变换。跨位置的信息混合主要由 self-attention 完成; 每个位置内部的特征重组、升维、非线性和降维主要由 FFN 完成。

¹²视频画面时间区间: 约 24:45–25:10; 字幕对应 “encoder 里面会分成很多很多的 block” “每一个 block 都是输入一排向量, 输出一排向量”。

4.2 位置编码不可省

Self-attention 本身对输入顺序不敏感。如果交换输入向量的顺序，attention 机制只看到一组向量，无法天然知道谁在前谁在后。Transformer 因此要把 positional encoding 加到 input embedding 上，让每个位置带有位置信息。

位置编码可以是固定的正弦余弦函数，也可以是学习出来的向量。无论形式如何，它解决的是同一个问题：让模型不只知道 token 内容，也知道 token 在序列中的位置。

4.3 本章小结

Transformer encoder 是一个可堆叠的向量序列变换器。它保留输入输出长度一致的形式，但通过多层 self-attention、FFN、残差和归一化，让每个输出向量包含全局上下文。位置编码负责把顺序信息注入原本 permutation-invariant 的 attention 计算。

5 一个 Encoder Block 的内部结构

老师用逐步展开的图解释一个 encoder block。第一步是 self-attention：输入一排向量，输出一排经过上下文混合的向量。接着不是直接把它传给下一层，而是先做 residual connection，也就是把 self-attention 的输入 a 和输出 b 相加：

$$u = a + b.$$

这里 a 是子层输入， $b = \text{SelfAttention}(a)$ 是子层输出， u 是残差相加后的结果。

残差连接的直觉是保留一条“原始信息和梯度都比较容易通过”的路径。深层网络如果每一层都必须完整重写表示，训练会困难；残差让每层更像是在学习一个修正量，而不是从零开始生成新表示。

5.1 Layer Normalization

Residual 之后接 LayerNorm。对一个向量

$$x = (x_1, x_2, \dots, x_K),$$

LayerNorm 会在同一个样本、同一个 token 的不同维度上计算均值和标准差：

$$m = \frac{1}{K} \sum_{i=1}^K x_i, \quad \sigma = \sqrt{\frac{1}{K} \sum_{i=1}^K (x_i - m)^2 + \epsilon},$$
$$x'_i = \frac{x_i - m}{\sigma}.$$

其中：

- K 是 hidden dimension；
- m 和 σ 都来自同一个向量内部的各个维度；
- ϵ 是避免除以零的小常数；

- 实际实现通常还会有可学习的缩放和平移参数 γ, β , 即 $\gamma x'_i + \beta$ 。

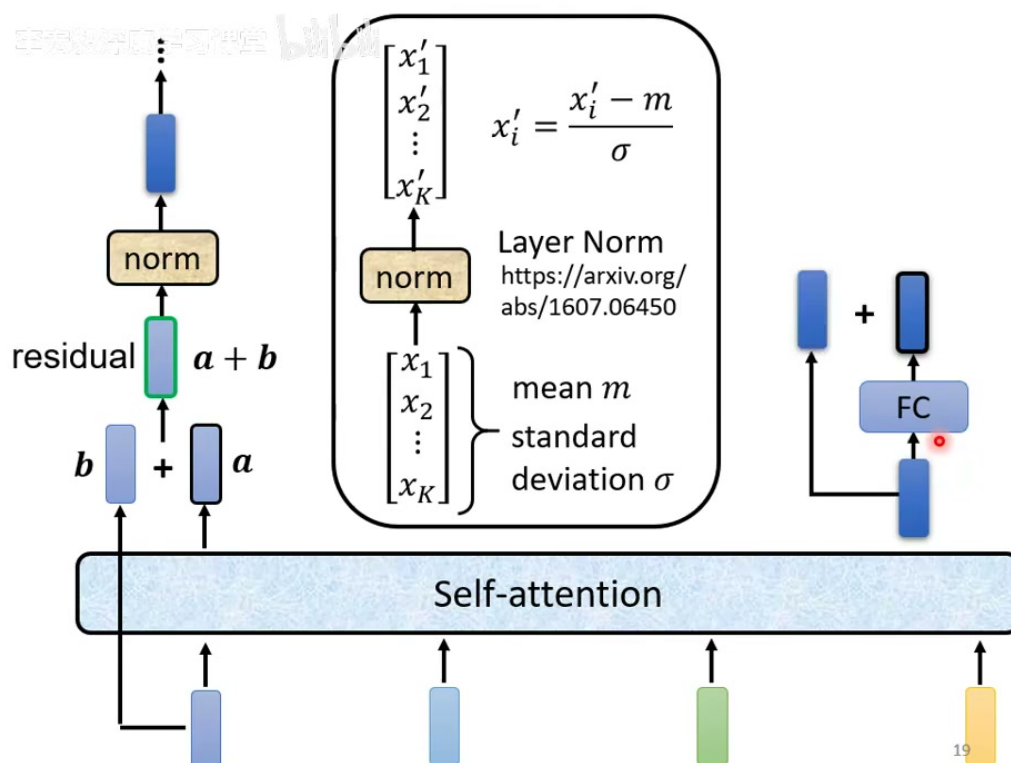


图 7: 一个 encoder block 中, self-attention 后接 residual 与 LayerNorm, 之后进入 FC/Feed Forward, 并再次使用 residual 结构。¹⁴

LayerNorm 和 BatchNorm 的统计方向不同

BatchNorm 通常对同一个 feature dimension、不同 example 计算统计量; LayerNorm 则对同一个 example 内部的不同 hidden dimensions 计算统计量。Transformer 常用 LayerNorm, 是因为序列长度、batch 组成和自回归生成场景会让 batch-level statistics 更不稳定。

5.2 Feed Forward Network 的角色

经过 self-attention 后, 每个 token 已经获得其他 token 的信息, 但这个混合后的向量还需要进一步非线性加工。FFN 通常写成

$$\text{FFN}(x) = W_2 \phi(W_1 x + b_1) + b_2,$$

其中 ϕ 可以是 ReLU、GELU 等非线性函数。这个 FFN 对每个位置独立套用同一组参数, 所以它不会改变序列长度。

一个常见 encoder block 可以抽象写成:

$$z = \text{LayerNorm}(x + \text{MHA}(x)),$$

¹⁴ 视频画面时间区间: 约 28:25–29:05; 字幕对应“每一个 dimension 减掉 mean 再除以 standard deviation”“fully connected network 这边也有 residual 的架构”。

$$h = \text{LayerNorm}(z + \text{FFN}(z)).$$

这里 MHA 是 multi-head attention, z 是 attention 子层后的表示, h 是整个 block 的输出。

需要注意：上面写法对应原始 Transformer 常见的 post-norm 形式。后来很多模型会改成 pre-norm, 也就是先 LayerNorm 再进入子层。两者训练稳定性不同, 后面课程和论文会进一步讨论。

5.3 本章小结

Encoder block 的内部并不只是 self-attention。一个完整 block 至少包含 self-attention、residual connection、LayerNorm、FFN, 以及 FFN 外面的 residual/normalization。Self-attention 负责跨位置沟通, FFN 负责位置内非线性变换, residual 和 LayerNorm 负责让深层堆叠可训练。

6 对照原始 Transformer Encoder 图

课程最后把刚才拆开的结构对回原始 Transformer 论文的 encoder 图。原图右侧可以看到：输入先经过 input embedding, 再加 positional encoding; 之后进入 N 次重复的 encoder block。每个 block 内部有两个子层：Multi-Head Attention 和 Feed Forward。每个子层旁边都有 Add & Norm。

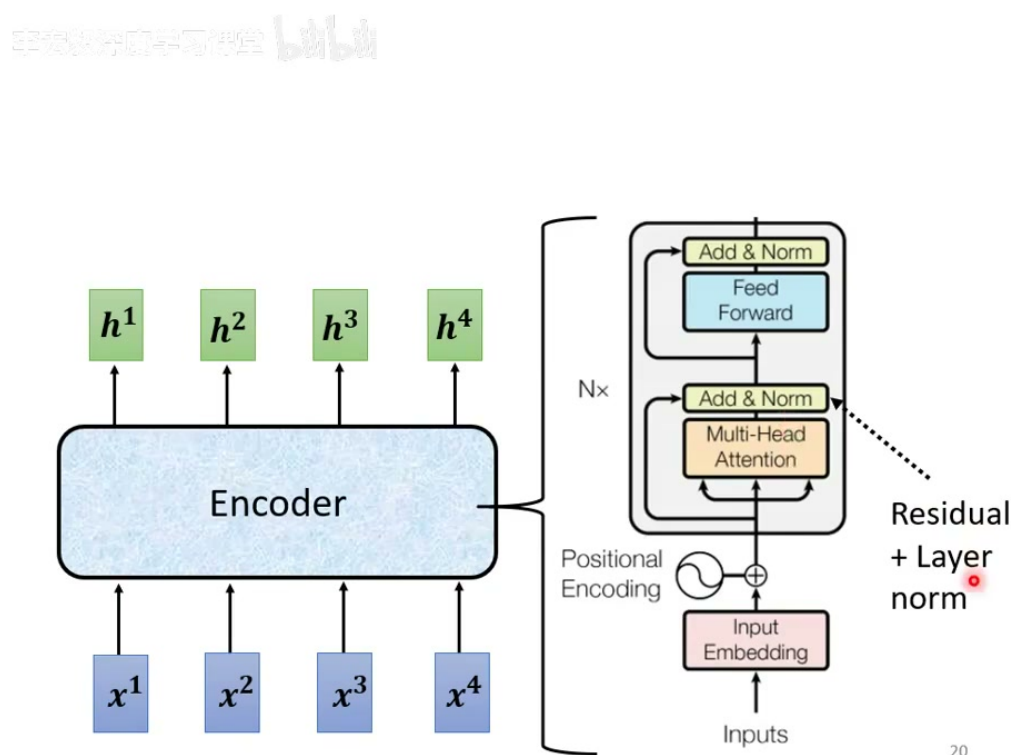


图 8: 原始 Transformer encoder 图中, Add & Norm 表示 residual connection 加 LayerNorm; Multi-Head Attention 对应 self-attention 子层, Feed Forward 对每个位置独立变换。¹⁶

这张图里有几个容易漏掉的细节：

¹⁶视频画面时间区间：约 29:40–30:25；字幕对应“positional information”“multi-head self attention”“Add and Norm 就是 residual 加 layer normalization”。

- 输入 embedding 和 positional encoding 是相加，不是串接；
- encoder block 重复 N 次，所以深度来自同一结构的堆叠；
- Multi-Head Attention 是 self-attention 的多头版本，每个 head 可以关注不同关系；
- Add & Norm 出现两次，分别包住 attention 子层和 FFN 子层；
- encoder 输出是一排向量，不是一个单一向量。

6.1 Multi-head 的意义

单头 attention 会把每个 token 对其他 token 的关系压进一组 attention weights。Multi-head attention 则把表示分成多个子空间，让不同 head 可以学不同关系：有的 head 可能关注局部搭配，有的关注长距离依赖，有的关注语法角色，有的关注实体对应。虽然不应把每个 head 都机械解释成某种固定语言学功能，但多头结构确实增加了模型同时表达多种关系的能力。

6.2 为什么这里讲 LayerNorm 而不是 BatchNorm

前一讲刚讲 Batch Normalization，所以这讲特别对比 LayerNorm。BatchNorm 的统计量依赖 batch 内其他样本；LayerNorm 的统计量来自单个样本的 hidden dimensions。对 Transformer 来说，LayerNorm 有几个优势：

- 推论时不需要维护 batch-level running mean/variance；
- 适合 batch size 变化较大的训练与推论场景；
- 自回归生成时一次可能只处理少量 token，BatchNorm 的 batch 统计不自然；
- 序列任务中每个位置的表示更适合在 hidden dimension 内稳定尺度。

这不是说 BatchNorm 不能用于 Transformer，而是原始架构和许多后续模型更偏向 LayerNorm。后来也有研究重新审视 normalization 的位置和形式，说明 Transformer 的细节设计并非不可改变。

6.3 本章小结

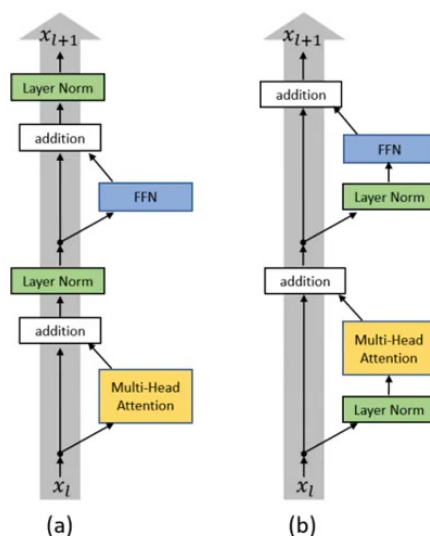
原始 Transformer encoder 图可以拆成清楚的四件事：embedding 加位置、multi-head self-attention、position-wise FFN、两处 Add & Norm。看懂这张图，就能把前面讲过的 self-attention 和后续会讲的 BERT、decoder、Seq2seq 训练连接起来。

7 Normalization 位置不是唯一答案

课程结尾提到：原始论文的结构不代表永远最优。比如 LayerNorm 应该放在 residual 之后，还是放在每个子层之前？这就是 post-norm 和 pre-norm 的差异。原始 Transformer 图更接近 post-norm；许多后来大模型为了训练更深网络，会采用 pre-norm 或其他变体。

To learn more

- On Layer Normalization in the Transformer Architecture
- <https://arxiv.org/abs/2002.04745>
- PowerNorm: Rethinking Batch Normalization in Transformers
- <https://arxiv.org/abs/2003.07845>



21

图 9: 课程给出的拓展阅读强调: Transformer 中 LayerNorm 的位置和 normalization 形式都可以研究, 原始设计不是唯一可行结构。¹⁸

7.1 Pre-norm 与 Post-norm 的直觉差异

用简化形式表示, post-norm 可以写成

$$x_{l+1} = \text{LN}(x_l + F(x_l)).$$

pre-norm 则可以写成

$$x_{l+1} = x_l + F(\text{LN}(x_l)).$$

两者都包含 residual 和 LayerNorm, 但梯度路径不同。pre-norm 中, 残差主路径更直接, 深层模型训练通常更稳定; post-norm 中, 归一化直接作用在相加后的输出上, 原始 Transformer 使用这种形式, 但堆很深时可能更难训练。

结构细节会影响可扩展性

Transformer 的大方向是 attention + FFN + residual + normalization, 但真正训练大模型时, LayerNorm 位置、激活函数、初始化、残差缩放、学习率计划都会影响稳定性。架构图看起来只是几个方块, 工程上每个方块的放置都可能决定模型能否顺利扩大。

¹⁸视频画面时间区间: 约 31:15–32:20; 字幕对应 “为什么 layer normalization 是放在那个地方呢” “原始论文的设计并不代表它是最好的”。

7.2 本章小结

原始 Transformer encoder 是理解后续模型的基准，但不是唯一答案。Normalization 的位置和形式仍然是可研究、可替换的设计空间。学架构时应该先掌握标准版，再知道哪些地方是历史选择、工程折衷或可调超参数。

8 总结与延伸

本讲完成了两个层次的铺垫。第一层是任务层：Transformer 是 Seq2seq model，而 Seq2seq 可以把语音识别、语音翻译、问答、摘要、句法分析、多标签分类和目标检测等任务放到同一个“输入序列到输出序列”的框架下。第二层是结构层：Transformer encoder 把一排输入向量变成一排上下文化向量，内部由 multi-head self-attention、FFN、residual connection、LayerNorm 和 positional encoding 组成。

可以把本讲压缩成下面这条链：

Seq2seq task → Encoder-Decoder framework → Transformer
→ Self-attention blocks → Residual + LayerNorm + FFN.

其中每一步都解决一个具体问题：

- Seq2seq 解决输入输出长度不固定的问题；
- encoder-decoder 分工解决“先理解输入，再产生输出”的问题；
- self-attention 解决序列中远距离位置互相参考的问题；
- multi-head 解决关系类型多样的问题；
- positional encoding 解决顺序信息缺失的问题；
- residual 和 LayerNorm 解决深层网络训练稳定性的问题；
- FFN 解决每个位置内部非线性加工的问题。

本讲最重要的理解

Transformer encoder 不是一个神秘黑盒，而是“向量序列变换”的可堆叠模块。它每层先让 token 之间互相交换信息，再对每个 token 的表示做非线性加工，并用残差与归一化维持训练稳定。这个结构既能作为完整 Seq2seq 的 encoder，也能单独成为 BERT 这类表示学习模型的核心。

下一讲自然会进入 decoder。Encoder 负责把输入序列变成表示；decoder 则要回答另一个难题：输出序列是一步一步生成的，模型在产生第 t 个 token 时不能偷看未来 token，同时还要能参考 encoder 的输出。于是 masked self-attention、cross-attention、teacher forcing、结束符号等机制都会登场。

8.1 拓展阅读

本讲画面中提到的几篇资料可以作为延伸：

- Attention Is All You Need: 原始 Transformer 架构；
- Grammar as a Foreign Language: 把 parsing 视为序列生成；
- On Layer Normalization in the Transformer Architecture: 讨论 LayerNorm 位置；
- PowerNorm: 重新思考 Transformer 中的 normalization。